CANTEENLY

Smart Canteen Management Ecosystem

SOFTWARE REQUIREMENTS SPECIFICATION

Version 2.0 | April 2026

Document Title	Canteenly SRS
Version	2.0 (Updated from v1.0 Draft)
Status	In Review
Date	April 2026
Classification	Confidential
Authors	Canteenly Team
Key Changes v2.0	Added DB schema, API contracts, real-time spec, auth decision, tech stack, error handling, testing requirements, dev phases, rate limiting, wastage clarification, i18n spec

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for Canteenly — a comprehensive smart canteen management ecosystem. It is intended for developers, designers, QA engineers, project managers, and stakeholders involved in the development and deployment of the system.

Version 2.0 incorporates all updates identified during the pre-development review, including the database schema, API endpoint contracts, real-time architecture decisions, authentication strategy, tech stack, development phases, and error-handling requirements.

1.2 Project Overview

Canteenly is a full-stack canteen operating system designed primarily for educational institutions (schools, colleges, and universities). It digitizes the entire canteen workflow — from student ordering and online payments to kitchen management and owner analytics — with an AI intelligence layer providing personalization, recommendations, and forecasting.

1.3 Scope

The system encompasses five core modules:

- Student App — Customer-facing mobile/web application
- Owner/Admin Dashboard — Management and analytics portal
- Point of Sales (POS) — Billing and walk-in order management
- Kitchen Display System (KDS) — Real-time kitchen operations
- Canteenly AI — Central intelligence and personalization engine (v1.1)

1.4 Definitions & Acronyms

SRS	Software Requirements Specification
POS	Point of Sales — terminal for walk-in billing
KDS	Kitchen Display System — real-time kitchen order screen
ERD	Entity Relationship Diagram
JWT	JSON Web Token — stateless authentication mechanism
SSE	Server-Sent Events
WebSocket	Full-duplex real-time communication protocol
API	Application Programming Interface
OTP	One-Time Password
RBAC	Role-Based Access Control
DXA	Unit used in Office documents (1440 DXA = 1 inch)
i18n	Internationalisation
FCM	Firebase Cloud Messaging
UPI	Unified Payments Interface (India)
PCI-DSS	Payment Card Industry Data Security Standard

1.5 Document Conventions

Requirements are labeled as FR (Functional Requirement) or NFR (Non-Functional Requirement), followed by module code and sequence number.

P1 — Critical	Must be in v1.0. System is non-functional without it.
P2 — High	Should be in v1.0. Significant user value.
P3 — Medium	Target for v1.1 if not in v1.0.
P4 — Low	Future enhancement / nice-to-have.

2. Overall System Description

2.1 Product Perspective

Canteenly is a new, standalone product. It integrates mobile and web applications, POS hardware terminals, kitchen display screens, a cloud backend, and an AI engine into a unified ecosystem. The system may optionally integrate with institutional ERP systems via REST APIs.

2.2 Confirmed Tech Stack

> **Note:** New in v2.0 — locking the tech stack before development begins prevents architectural drift.

Student App	React Native (iOS + Android) + PWA fallback
Admin Dashboard	Next.js (React) — web browser
POS Terminal	Next.js — tablet / desktop browser (Chrome kiosk mode)
KDS	Next.js — dedicated screen (Chrome kiosk mode)
Backend	Node.js + Express (REST API microservices)
Primary DB	PostgreSQL (Supabase-hosted for v1.0)
Cache	Redis (session tokens, real-time slot availability)
Analytics DB	MongoDB (AI / analytics data — v1.1)
Auth	Supabase Auth (handles OTP, JWT, SSO/LDAP)
Real-time	WebSockets via Socket.io (KDS stream + order status)
Payments	Razorpay (India primary — cards, UPI, net banking, wallets)
Push Notifs	Firebase Cloud Messaging (FCM)
File Storage	Supabase Storage (menu item images)
Deployment	Docker + Railway / Render (v1.0); Kubernetes (v2.0)
API Docs	OpenAPI 3.0 (auto-generated via Swagger)
i18n	react-i18next (frontend); Accept-Language header (backend)

2.3 System Architecture Overview

The Canteenly ecosystem is composed of five interconnected components communicating through a central cloud API layer. All inter-module communication is via REST endpoints except for the real-time KDS stream and order status updates, which use WebSockets (Socket.io).

- Student App (iOS / Android / PWA)
- Owner/Admin Dashboard (Next.js Web App)
- Point of Sales Terminal (Next.js Tablet/Desktop App)
- Kitchen Display System (Next.js Dedicated Screen)
- Canteenly AI Engine (Node.js Microservice — v1.1)

2.4 User Classes and Characteristics

User Class	Description	Tech Level	Module Access
Student / Customer	Places orders, browses menu, makes payments	Low–Medium	Student App
Canteen Owner	Manages menu, monitors analytics, views reports	Low–Medium	Admin Dashboard
Admin Staff	Operates POS, handles walk-in billing	Medium	POS Terminal
Kitchen Staff	Views and acts on orders via KDS	Low	KDS
System Administrator	Configures system, manages accounts, API access	High	All modules

2.5 Operating Environment

- Student App: iOS 14+, Android 9+, modern web browsers (Chrome, Safari, Firefox)
- Admin Dashboard: Web browser (Chrome / Edge / Firefox), desktop or tablet
- POS Terminal: Tablet (Android / iPad) or desktop browser
- KDS: Dedicated 10"+ display, Chrome browser in kiosk mode
- Backend: Supabase (PostgreSQL + Auth + Storage) + Node.js hosted on Railway / Render

2.6 Assumptions and Dependencies

- Stable internet connectivity is available at the canteen premises.
- Razorpay account and API keys will be provisioned before Student App development begins.
- Supabase project will be set up and schema migrated before any coding begins.
- Wearable device APIs (watchOS / Wear OS) are accessible for AI integration (v1.1).
- The institution can provide student authentication data (student IDs, emails) for SSO.

3. Development Phases (MVP Scope)

Note: New in v2.0 — defines the MVP build order agreed during pre-development review.

Phase	Module	Deliverables	Unlocks
1	Foundation	DB schema + migrations, API contracts, Supabase project setup, auth strategy, Socket.io config	All subsequent phases
2	Admin Dashboard	Menu CRUD, category management, pricing, availability toggle, time slot configuration	Menu data for POS + Student App
3	KDS	Real-time order display, status updates, WebSocket stream, audio alerts	Testable order flow end-to-end
4	POS	Walk-in billing, cash/card/UPI, KDS sync, offline queue	Full canteen operation test
5	Student App	Auth, menu browse, pre-order + immediate order, Razorpay payment, order tracking push notifications	Complete v1.0 system
6	AI + Analytics	Demand forecasting, personalised recommendations, wastage alerts, sales reports	Requires 30+ days of order data

4. Functional Requirements

4.1 Student App

The Student App is the primary customer-facing interface allowing students to browse menus, place orders, and complete payments.

4.1.1 User Registration & Authentication

ID	Requirement	Priority	Notes
FR-SA-01	Students shall register using institutional email or phone with OTP verification.	P1	OTP via Supabase Auth
FR-SA-02	Students shall log in using email/password or OTP.	P1	Supabase Auth handles both
FR-SA-03	System shall support SSO via institutional LDAP/Active Directory credentials.	P2	Supabase LDAP integration
FR-SA-04	Students shall reset password via email or OTP.	P1	
FR-SA-05	All auth tokens shall be JWT (access + refresh). Access token TTL: 15 min. Refresh: 7 days.	P1	New in v2.0

4.1.2 Menu Browsing

ID	Requirement	Priority	Notes
FR-SA-06	Students shall browse the full menu with items categorised by food type.	P1	
FR-SA-07	Each item shall display name, description, price, availability status, and prep time.	P1	
FR-SA-08	Students shall search and filter by category, dietary preference (veg/non-veg), and price.	P2	
FR-SA-09	System shall display real-time availability — items marked unavailable when out of stock.	P1	WebSocket push from admin toggle

4.1.3 Ordering with Time Slots

ID	Requirement	Priority	Notes
FR-SA-10	Students shall pre-order by selecting a future pickup time slot.	P1	Slot capacity enforced server-side
FR-SA-11	Students shall place immediate orders (real-time ordering).	P1	
FR-SA-12	Pre-order window: up to 24 hours ahead. Slots that reach max_orders capacity shall be hidden.	P1	New in v2.0 — closes SRS gap

ID	Requirement	Priority	Notes
FR-SA-13	System shall allow customisation of order items (e.g. extra spicy, no onion).	P2	Free-text field, max 200 chars
FR-SA-14	Students shall view and manage cart before placing the order.	P1	
FR-SA-15	Students shall receive order confirmation with estimated preparation time.	P1	FCM push notification

4.1.4 Online Payment

ID	Requirement	Priority	Notes
FR-SA-16	System shall support online payment via credit/debit cards, UPI, and net banking via Razorpay.	P1	
FR-SA-17	System shall support canteen wallet (pre-loaded balance).	P2	
FR-SA-18	Payment transactions shall be PCI-DSS compliant via Razorpay gateway.	P1	
FR-SA-19	Students shall receive a digital receipt (push + email) on successful payment.	P1	
FR-SA-20	Refunds for cancelled orders shall be auto-initiated via Razorpay API within 24 hours.	P2	New in v2.0 — defines refund flow
FR-SA-21	Refunds are only eligible if order status is pending at time of cancellation.	P1	New in v2.0 — closes SRS gap

4.1.5 Order Tracking

ID	Requirement	Priority	Notes
FR-SA-22	Students shall track real-time preparation status: Received → Preparing → Ready.	P1	Driven by KDS status updates via WebSocket
FR-SA-23	System shall send FCM push for each status change.	P1	
FR-SA-24	Students shall view complete order history (paginated, newest first).	P2	
FR-SA-25	Students shall rate and review items after order completion.	P3	

4.2 Owner / Admin Dashboard

Provides canteen operators with tools to manage their menu, monitor performance, and analyse business data.

4.2.1 Menu Management

ID	Requirement	Priority	Notes
FR-AD-01	Owners shall add, edit, and delete menu items with pricing and descriptions.	P1	
FR-AD-02	Owners shall toggle item availability in real time (broadcasts to Student App via WebSocket).	P1	New: real-time broadcast spec
FR-AD-03	Owners shall update item quantity_available field; system auto-marks unavailable at zero.	P1	New in v2.0
FR-AD-04	Owners shall configure time-slot availability for specific items.	P2	
FR-AD-05	Owners shall set up combo offers and promotional pricing.	P2	

4.2.2 Sales & Revenue Analytics

ID	Requirement	Priority	Notes
FR-AD-06	Dashboard shall display daily, weekly, and monthly revenue reports.	P1	
FR-AD-07	System shall provide item-level sales analysis (best sellers, slow movers).	P1	
FR-AD-08	Owners shall export reports in PDF and Excel formats.	P2	
FR-AD-09	Dashboard shall display order volume trends and peak-hour analytics.	P2	

4.2.3 Food Wastage Monitoring

Note: Clarified in v2.0 — wastage is calculated as qty_prepared (admin input) minus qty_sold (from orders). Admin submits daily qty_prepared via POST /wastage/log.

ID	Requirement	Priority	Notes
FR-AD-10	Admin shall submit end-of-day qty_prepared per item via the wastage log endpoint.	P1	New in v2.0 — closes inventory gap
FR-AD-11	System shall compute qty_wasted = qty_prepared − qty_sold automatically.	P1	New in v2.0
FR-AD-12	Owners shall receive alerts when wastage exceeds a configurable threshold.	P2	
FR-AD-13	System shall provide wastage trend reports to optimise production.	P2	

4.3 Point of Sales (POS)

ID	Requirement	Priority	Notes
FR-POS-01	POS staff shall browse menu and add items to a customer bill.	P1	Reads from same menu API
FR-POS-02	POS shall support cash, card, and UPI payments.	P1	
FR-POS-03	POS shall auto-sync all walk-in orders to KDS via WebSocket (event: order.new).	P1	Order source=pos
FR-POS-04	POS shall display active online orders alongside walk-in orders.	P1	GET /pos/orders/active
FR-POS-05	POS shall generate and optionally print a receipt per transaction.	P2	
FR-POS-06	POS shall support discount application with admin PIN authorisation.	P3	
FR-POS-07	POS shall support offline mode — transactions queue locally and sync on reconnect (max 200 orders).	P2	POST /pos/orders/offline-sync
FR-POS-08	During POS offline mode, KDS will not receive new POS orders until sync completes. KDS shall display a warning banner when sync is pending.	P1	New in v2.0 — closes sync conflict gap

4.4 Kitchen Display System (KDS)

ID	Requirement	Priority	Notes
FR-KDS-01	KDS shall display all active orders (online and walk-in) in real time via WebSocket.	P1	
FR-KDS-02	Orders prioritised by: (1) pickup time slot ASC, (2) created_at ASC. Walk-in vs pre-order tiebreaker: earlier slot wins; if same slot, created_at.	P1	New in v2.0 — closes prioritisation gap
FR-KDS-03	Kitchen staff shall update status: Received → Preparing → Ready.	P1	PATCH /kds/orders/:id/status
FR-KDS-04	Status updates shall trigger FCM push to student app (event: order.status).	P1	
FR-KDS-05	KDS shall display elapsed time per order and visually flag orders overdue by > 5 min.	P2	
FR-KDS-06	KDS shall support order bumping (mark done, remove from screen).	P1	
FR-KDS-07	KDS shall support audio alerts for new incoming orders.	P2	
FR-KDS-08	KDS shall display a sync-pending warning banner when POS is in offline mode.	P1	New in v2.0

4.5 Canteenly AI Engine (v1.1)

ID	Requirement	Priority	Notes
FR-AI-01	AI engine shall generate personalised menu recommendations from order history.	P1 (v1.1)	Requires 30+ days data
FR-AI-02	System shall connect with Apple Watch / Wear OS for contextual recommendations (explicit user consent required).	P2 (v1.1)	
FR-AI-03	AI engine shall provide demand forecasting for daily production quantities.	P1 (v1.1)	
FR-AI-04	AI shall predict peak ordering periods and suggest staffing adjustments.	P2 (v1.1)	
FR-AI-05	AI shall learn user preferences over time and improve accuracy.	P1 (v1.1)	
FR-AI-06	AI engine input/output contract: input = {user_id, canteen_id, time_of_day, order_history[]}; output = {recommended_item_ids[], confidence_scores[]}	P1 (v1.1)	New in v2.0 — closes AI spec gap

5. Non-Functional Requirements

5.1 Performance

ID	Requirement	Target / Detail	Priority
NFR-PERF-01	API response time — order placement	< 2 seconds (95th percentile)	P1
NFR-PERF-02	KDS order update latency (WebSocket)	< 1 second	P1
NFR-PERF-03	Dashboard analytics page load	< 3 seconds	P1
NFR-PERF-04	Concurrent users supported	500+ simultaneous	P1
NFR-PERF-05	Payment gateway response	< 5 seconds	P1
NFR-PERF-06	Load testing required before go-live	Verify NFR-PERF-04 with k6 or Locust	P1 — New v2.0

5.2 Security

ID	Requirement	Target / Detail	Priority
NFR-SEC-01	All data in transit encrypted	TLS 1.2+ mandatory	P1
NFR-SEC-02	Password storage	bcrypt or Argon2 (salted)	P1
NFR-SEC-03	Payment data	PCI-DSS via Razorpay	P1
NFR-SEC-04	Access control	RBAC across all modules	P1
NFR-SEC-05	API authentication	JWT (Supabase Auth)	P1
NFR-SEC-06	Admin/POS audit trail	All actions logged with actor_id + timestamp	P1
NFR-SEC-07	API rate limiting	Max 60 req/min per IP; max 10 order attempts/min per user	P1 — New v2.0
NFR-SEC-08	Input validation	All API inputs validated server-side; SQL injection prevention via parameterised queries	P1 — New v2.0

5.3 Reliability & Availability

ID	Requirement	Target / Detail	Priority
NFR-REL-01	System uptime SLA	99.5% (excl. planned maintenance)	P1
NFR-REL-02	POS + KDS offline mode	Local caching during network outages	P1
NFR-REL-03	Data backup	Automated daily + point-in-time recovery	P1
NFR-REL-04	WebSocket reconnect	Auto-reconnect with exponential backoff on disconnect	P1 — New v2.0

5.4 Scalability

ID	Requirement	Target / Detail	Priority
NFR-SCL-01	Backend containerisation	Docker + horizontal scaling	P1
NFR-SCL-02	Multi-canteen support	Planned for v2.0	P3
NFR-SCL-03	Database partitioning	For large historical data volumes	P2

5.5 Usability

ID	Requirement	Target / Detail	Priority
NFR-USE-01	Student App task completion	90%+ in usability testing	P1
NFR-USE-02	KDS touch targets	Operable with kitchen gloves (min 48×48px touch target)	P1
NFR-USE-03	Primary flows reachable	Within 3 taps/clicks from home screen	P1
NFR-USE-04	Internationalisation (i18n)	English + Hindi at launch; react-i18next; Accept-Language header on backend	P1 — New v2.0
NFR-USE-05	Accessibility	WCAG 2.1 AA for web interfaces	P2 — New v2.0

5.6 Maintainability

ID	Requirement	Target / Detail	Priority
NFR-MNT-01	API documentation	OpenAPI 3.0 / Swagger auto-generated	P1
NFR-MNT-02	Zero-downtime deployments	Blue-green or rolling updates	P2
NFR-MNT-03	Health-check endpoints	GET /health on all services	P1
NFR-MNT-04	Code coverage	Minimum 70% unit test coverage before go-live	P1 — New v2.0

6. Error Handling Requirements

Note: New section in v2.0 — defines expected system behaviour for all critical failure scenarios.

Scenario	Expected Behaviour	User-Facing Message
Payment fails mid-order	Order status set to payment_failed. Inventory not decremented. Student prompted to retry.	"Payment failed. Please try again or use a different method."
Payment succeeds but webhook delayed	Order marked pending_payment. Webhook reconciles within 60s. No duplicate order created.	"Your payment is being confirmed. We will notify you shortly."
KDS goes offline	POS continues in offline mode. Orders queue locally. KDS shows last-known state until reconnect.	KDS shows: "Connection lost. Displaying cached orders."
AI engine unreachable	API returns 200 with empty recommendations array. No error surfaced to student.	Recommendations section hidden silently.
Time slot fully booked	Slot removed from available list. POST /orders returns 409 Conflict if race condition.	"This time slot is now full. Please choose another."
OTP not received	Resend available after 30s. Max 5 OTP attempts per phone/email per hour.	"Didn't receive OTP? Resend in {n}s"
Offline POS sync fails	Retry 3 times with exponential backoff. If all fail, alert admin and hold queue.	Staff sees: "Sync failed. Contact administrator."

7. External Interface Requirements

7.1 Payment Gateway — Razorpay

Razorpay is the selected gateway for v1.0 (India deployment). Integration supports cards, UPI, net banking, and wallets. Webhooks used for real-time payment status updates. Signature verification mandatory on all webhook events.

7.2 Push Notification Service — FCM

Firebase Cloud Messaging (FCM) handles iOS and Android push notifications. Notification types: order confirmation, order status change (Received / Preparing / Ready), promotional offers, and low-balance wallet alerts.

7.3 Real-Time Communication — Socket.io

> **Note:** New in v2.0 — WebSocket technology selected. Socket.io chosen for auto-reconnect, rooms, and browser compatibility.

Event: order.new	Emitted to room canteen:{id} when any order (POS or Student App) is placed. KDS subscribes.
Event: order.status	Emitted to room order:{id} when KDS updates status. Student App subscribes per-order.
Event: menu.availability	Emitted to room canteen:{id} when admin toggles item availability. Student App re-renders menu.
Reconnect policy	Exponential backoff: 1s, 2s, 4s, 8s, max 30s. Client shows offline banner after 3 failed attempts.

7.4 Wearable Device Integration (v1.1)

The Canteenly AI module will integrate with watchOS and Wear OS APIs to receive contextual data (activity level, location) for personalised food recommendations. Explicit user consent required. Data not stored beyond the session.

7.5 Institutional ERP / LDAP

Optional SSO via LDAP/Active Directory via Supabase Auth LDAP provider. Student enrollment data may be synced via REST API or CSV import.

8. Database Schema

Note: New section in v2.0. Primary database: PostgreSQL (Supabase). All tables use UUID primary keys. Timestamps are UTC.

8.1 users

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	Unique user identifier
name	VARCHAR(100)	NOT NULL		Full name
email	VARCHAR(255)	UNIQUE NOT NULL		Institutional or personal email
phone	VARCHAR(20)	UNIQUE		Mobile number for OTP
role	VARCHAR(20)	NOT NULL DEFAULT student		Enum: student, owner, staff, kitchen, sysadmin
institution_id	VARCHAR(100)			Student ID / roll number
password_hash	TEXT			Managed by Supabase Auth
is_active	BOOLEAN	DEFAULT true		Soft-disable without deletion
created_at	TIMESTAMPTZ	DEFAULT now()		

8.2 canteens

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
name	VARCHAR(150)	NOT NULL		Canteen display name
address	TEXT			Physical location
owner_id	UUID	NOT NULL	FK → users.id	Canteen owner reference
is_active	BOOLEAN	DEFAULT true		
created_at	TIMESTAMPTZ	DEFAULT now()		

8.3 categories

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	

Column	Type	Constraints	PK/FK	Description
canteen_id	UUID	NOT NULL	FK → canteens.id	
name	VARCHAR(100)	NOT NULL		e.g. Beverages, Snacks, Meals
sort_order	INT	DEFAULT 0		Display ordering
is_active	BOOLEAN	DEFAULT true		

8.4 menu_items

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
canteen_id	UUID	NOT NULL	FK → canteens.id	
category_id	UUID	NOT NULL	FK → categories.id	
name	VARCHAR(150)	NOT NULL		
description	TEXT			
price	DECIMAL(10,2)	NOT NULL		In INR
quantity_available	INT	DEFAULT 0		Decremented on order; 0 = auto unavailable
is_available	BOOLEAN	DEFAULT true		Manual toggle by admin
is_veg	BOOLEAN	DEFAULT true		
prep_time_mins	INT	DEFAULT 10		Displayed to student
image_url	TEXT			Supabase Storage URL
updated_at	TIMESTAMPTZ	DEFAULT now()		

8.5 time_slots

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
canteen_id	UUID	NOT NULL	FK → canteens.id	
start_time	TIME	NOT NULL		e.g. 12:00:00
end_time	TIME	NOT NULL		e.g. 12:30:00

Column	Type	Constraints	PK/FK	Description
max_orders	INT	NOT NULL		Capacity per slot
is_active	BOOLEAN	DEFAULT true		

8.6 orders

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
user_id	UUID		FK → users.id	NULL for anonymous walk-in
canteen_id	UUID	NOT NULL	FK → canteens.id	
time_slot_id	UUID		FK → time_slots.id	NULL for immediate orders
order_type	VARCHAR(20)	NOT NULL		Enum: immediate, preorder
source	VARCHAR(10)	NOT NULL		Enum: app, pos
status	VARCHAR(20)	NOT NULL		Enum: pending_payment, received, preparing, ready, completed, cancelled, payment_failed
payment_status	VARCHAR(20)	NOT NULL		Enum: pending, paid, refunded, failed
total_amount	DECIMAL(10,2)	NOT NULL		
pickup_time	TIMESTAMPTZ			For preorders
created_at	TIMESTAMPTZ	DEFAULT now()		

8.7 order_items

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
order_id	UUID	NOT NULL	FK → orders.id	CASCADE DELETE
menu_item_id	UUID	NOT NULL	FK → menu_items.id	
quantity	INT	NOT NULL DEFAULT 1		
unit_price	DECIMAL(10,2)	NOT NULL		Snapshot at time of order
customization	TEXT			Free-text, max 200 chars

8.8 payments

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
order_id	UUID	NOT NULL UNIQUE	FK → orders.id	One payment per order
gateway	VARCHAR(20)	DEFAULT razorpay		
gateway_txn_id	VARCHAR(100)			Razorpay payment_id
gateway_order_id	VARCHAR(100)			Razorpay order_id
method	VARCHAR(20)			card, upi, netbanking, wallet, cash
amount	DECIMAL(10,2)	NOT NULL		
status	VARCHAR(20)	NOT NULL		pending, paid, refunded, failed
paid_at	TIMESTAMPTZ			
refunded_at	TIMESTAMPTZ			

8.9 wastage_logs

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	
canteen_id	UUID	NOT NULL	FK → canteens.id	
menu_item_id	UUID	NOT NULL	FK → menu_items.id	
log_date	DATE	NOT NULL		End-of-day log
qty_prepared	INT	NOT NULL		Admin input via /wastage/log
qty_sold	INT	NOT NULL		Computed from orders
qty_wasted	INT	GENERATED ALWAYS AS (qty_prepared - qty_sold) STORED		Auto-computed

8.10 audit_logs

Column	Type	Constraints	PK/FK	Description
id	UUID	NOT NULL	PK	

Column	Type	Constraints	PK/FK	Description
actor_id	UUID	NOT NULL	FK → users.id	Who performed the action
action	VARCHAR(100)	NOT NULL		e.g. menu.update, order.cancel, discount.apply
entity_type	VARCHAR(50)	NOT NULL		e.g. order, menu_item
entity_id	UUID	NOT NULL		
payload	JSONB			Before/after snapshot
created_at	TIMESTAMPTZ	DEFAULT now()		

9. API Endpoints

Note: New section in v2.0. Base URL: https://api.canteenly.app/v1. All authenticated endpoints require Authorization: Bearer {jwt}. Rate limit: 60 req/min per IP. Responses follow {success, data, error} envelope.

9.1 Authentication

Method	Path	Description	Auth	Notes
POST	/auth/register	Register via email/phone. Sends OTP.	Public	Body: email|phone, name, password
POST	/auth/verify-otp	Verify OTP → returns JWT pair.	Public	Body: email|phone, otp
POST	/auth/login	Email + password login.	Public	Body: email, password
POST	/auth/refresh	Exchange refresh for access token.	Refresh token	Body: refresh_token
POST	/auth/logout	Revoke refresh token.	Bearer	

9.2 Menu Management

Method	Path	Description	Auth	Notes
GET	/canteen/:id/menu	Full menu with categories. Used by all clients.	Public	Includes is_available, quantity
POST	/canteen/:id/menu	Create menu item.	owner	Body: name, price, category_id, is_veg, prep_time_mins, quantity_available
GET	/menu/:id	Single item detail.	Public	
PATCH	/menu/:id	Edit item fields.	owner	Partial update allowed
PATCH	/menu/:id/availability	Toggle availability + update quantity. Broadcasts event: menu.availability via WebSocket.	owner	Body: is_available, quantity_available
DELETE	/menu/:id	Soft-delete item (is_active=false).	owner	
GET	/canteen/:id/categories	List categories.	Public	
POST	/canteen/:id/categories	Create category.	owner	Body: name, sort_order
PATCH	/categories/:id	Update category.	owner	

Method	Path	Description	Auth	Notes
DELETE	/categories/:id	Soft-delete category.	owner	

9.3 Time Slots

Method	Path	Description	Auth	Notes
GET	/canteen/:id/slots	All time slots with current order count.	owner	
GET	/canteen/:id/slots/available	Slots with remaining capacity for today.	Public	Used by Student App order screen
POST	/canteen/:id/slots	Create slot.	owner	Body: start_time, end_time, max_orders
PATCH	/slots/:id	Update slot.	owner	
DELETE	/slots/:id	Deactivate slot.	owner	

9.4 Orders — Student App

Method	Path	Description	Auth	Notes
POST	/orders	Place order.	student	Body: canteen_id, items[], time_slot_id?, order_type
GET	/orders/:id	Single order detail.	student/owner/staff	Includes items, status, payment
GET	/orders/my	Student order history (paginated).	student	Query: ?page=&limit=
POST	/orders/:id/cancel	Cancel if status=pending. Initiates refund.	student	Returns 409 if not cancellable
POST	/orders/:id/rate	Submit item ratings post-completion.	student	Body: ratings[{item_id, score, comment}]

9.5 Orders — POS

Method	Path	Description	Auth	Notes
POST	/pos/orders	Create walk-in order. source=pos. Auto-pushes to KDS.	staff	Body: items[], payment_method, canteen_id
GET	/pos/orders/active	All active orders (online + walk-in) for canteen.	staff	

Method	Path	Description	Auth	Notes
POST	/pos/orders/offline-sync	Batch sync queued offline orders.	staff	Body: orders[] (array of offline order payloads)
POST	/pos/orders/:id/discount	Apply discount. Requires admin PIN.	staff	Body: discount_type, amount, admin_pin

9.6 Payments

Method	Path	Description	Auth	Notes
POST	/payments/initiate	Create Razorpay order. Returns gateway order_id.	student/staff	Body: order_id
POST	/payments/webhook	Razorpay webhook handler. Verifies HMAC signature.	Public (Razorpay)	Signature verified server-side
GET	/payments/:order_id	Payment status for an order.	student/owner/staff	
POST	/payments/:order_id/refund	Initiate refund for cancelled order.	owner/system	Auto-triggered on order cancel

9.7 KDS

Method	Path	Description	Auth	Notes
GET	/kds/orders	Active orders sorted by priority (slot ASC, created_at ASC).	kitchen	Initial KDS load
PATCH	/kds/orders/:id/status	Update status. Broadcasts event: order.status.	kitchen	Body: status (received|preparing|ready|bumped)
GET	/kds/orders/:id	Full order detail for expanded view.	kitchen	

WebSocket events (Socket.io):

order.new → room: canteen:{id}	Emitted when any order (app or POS) is placed. KDS subscribes.
order.status → room: order:{id}	Emitted when KDS updates order status. Student App subscribes per-order.

menu.availability → room: canteen:{id}	Emitted when admin toggles item. Student App re-renders menu.

9.8 Analytics

Method	Path	Description	Auth	Notes
GET	/analytics/revenue	Revenue summary.	owner	Query: ?period=daily|weekly|monthly&from=&to=
GET	/analytics/items	Per-item sales count and revenue.	owner	Best sellers + slow movers
GET	/analytics/peak-hours	Order volume by hour of day.	owner	
GET	/analytics/wastage	Wastage summary.	owner	Query: ?from=&to=&item_id=
POST	/wastage/log	Admin submits end-of-day qty_prepared.	owner	System computes qty_wasted automatically
GET	/analytics/export	Download PDF or Excel report.	owner	Query: ?type=revenue|wastage&format=pdf|xlsx

9.9 System

Method	Path	Description	Auth	Notes
GET	/health	Service health check.	Internal	Returns DB, Redis, WebSocket status
GET	/version	API version and build info.	Internal	

10. Testing Requirements

Note: New section in v2.0. Testing must be written alongside features, not after.

Type	Scope	Tool	Coverage Target
Unit tests	All API route handlers, business logic (order pricing, slot capacity, wastage calc)	Jest (Node.js)	70% minimum
Integration tests	DB operations, payment webhook flow, WebSocket event emission	Jest + Supertest	Key flows 100%
E2E tests	Student App order flow, POS walk-in + KDS update, Admin menu edit + real-time propagation	Playwright	Critical paths
Load tests	500 concurrent users on POST /orders and GET /canteen/:id/menu	k6	Before go-live
Security tests	JWT validation, RBAC enforcement, SQL injection, rate limiting	OWASP ZAP	Before go-live

11. System Constraints & Limitations

- v1.0 supports single-canteen deployment. Multi-canteen is planned for v2.0.
- Wearable integration limited to Apple Watch and Wear OS in v1.1.
- AI demand forecasting requires minimum 30 days of historical order data.
- Offline POS mode supports max 200 queued orders before mandatory sync.
- Raw ingredient inventory tracking is not in v1.0 (planned v2.0).
- The wastage module requires admin to manually input qty_prepared daily; it does not auto-track production quantities.

12. Data Requirements

12.1 Data Retention

Order data	5 years minimum (compliance + analytics)
Payment transaction data	7 years (financial compliance)
User profile data	Retained until account deletion + 90-day grace period
KDS display logs	90 days
Audit logs	2 years
Wastage logs	3 years

12.2 Data Privacy

The system shall comply with applicable data protection regulations (IT Act 2000 for India; GDPR for international deployments). Personal data shall not be shared with third parties except payment processors (Razorpay) and notification providers (FCM). Students may export or delete their personal data on request (data portability / right to erasure).

Wearable data (v1.1): Activity and location data received from wearables shall not be stored server-side. It is used in-memory per recommendation request only and discarded after response.

13. Appendices

Appendix A — Revision History

Version	Date	Author	Description
1.0	April 13, 2026	Canteenly Team	Initial SRS draft
2.0	April 2026	Canteenly Team + Claude	Added: DB schema (Section 8), API contracts (Section 9), tech stack (Section 2.2), dev phases (Section 3), error handling (Section 6), testing requirements (Section 10), real-time spec (WebSocket/Socket.io), auth strategy (Supabase), rate limiting NFR, wastage calculation clarification, i18n spec, order status enum, refund policy, KDS prioritisation tiebreaker, AI input/output contract, offline POS conflict resolution.

Appendix B — Open Issues

#	Issue	Owner	Target	Status
1	Wearable API access approval from Apple and Google pending.	Canteenly Team	v1.1	Open
2	Multi-language scope (additional regional languages beyond Hindi) to be confirmed.	Canteenly Team	v2.0	Open
3	Razorpay account KYC and webhook URL registration.	Canteenly Team	Phase 5	Open
4	Institution LDAP endpoint access for SSO.	Institution IT / Canteenly	Phase 2	Open

Appendix C — Future Enhancements (v2.0 Roadmap)

- Multi-canteen / multi-location support under single admin account.
- Raw ingredient inventory and supplier management.
- Nutritional information and calorie tracking per menu item.
- QR code-based contactless table ordering.
- Loyalty points and reward program for students.
- Advanced AI meal planning based on dietary goals and health data from wearables.
- Kubernetes deployment for production-grade horizontal scaling.